

**PONTIFICIA UNIVERSIDAD CATÓLICA MADRE Y MAESTRA
FACULTAD DE CIENCIAS E INGENIERÍA
ESCUELA DE INGENIERÍA EN COMPUTACIÓN Y TELECOMUNICACIONES**



Fundamentos de Sistemas computacionales

5176

**Reporte #3
Laboratorio de FPGA – Lab. 5**

ESTUDIANTE:
CARLOS DAVID JORGE TAVERAS
GIAN ALEJANDRO CUNAZZA TORRES

DOCENTE:
PAMELA HERNÁNDEZ ESTÉVEZ.

FECHA DE ENTREGA:
14/7/2026

**Santiago de los Caballeros,
República Dominicana**

Índice.

Introducción	3
Objetivos	4
Marco teórico	4
Procedimiento de desarrollo	5
Conclusión	9

Introducción

En este laboratorio de FPGA, hemos logrado hacer una calculadora básica, capaz de sumar, restar y multiplicar. Aunque, el objetivo principal era crear un sumador, nos entro la curiosidad como seria restar, y multiplicar. Explicaremos a detalle los procesos de desarrollo, los materiales utilizados y el aprendizaje obtenido a base de esto. El programa consiste en utilizar los switches de la placa booleana para definir los valores a sumar, restar o multiplicar, y los botones para permitir las acciones aritméticas.

Objetivos

El objetivo principal del laboratorio nuevamente involucra el concepto de entender la programación de bajo nivel, en este caso; FPGA. Aquí, nuevamente utilizamos “Verilog” con una placa booleana que nos provee el campus, específicamente el laboratorio de Telecomunicaciones.

A diferencia del laboratorio 4, el cual consistía en mostrar “Hola” en la pantalla de 7 segmentos, nuestro objetivo principal es hacer un sumador, el cual permita a un usuario utilizar los botones o switches del hardware para sumar dos valores e imprimir el resultado en la pantalla de 7 segmentos. Cabe anotar que, los valores seleccionados a sumar deben ser igualmente mostrados en la pantalla de 7 segmentos.

Marco teórico

El editor de texto utilizado para la programación fue el “Zed Editor”, utilizando un plugin el cual está en el Marketplace de este. Como se menciona antes, el lenguaje principal fue “Verilog”. Para la compilación del código fue requerido utilizar Docker. El propósito fue reducir el uso del espacio en el disco, las herramientas comunes proporcionadas por AMD pueden llegar a ocupar 90 Gb en su totalidad. Con Docker he logrado ahorrarme 82 Gb, utilizando solo las herramientas que necesito.

Para la compilación, fue utilizado “Yosys” junto a otras herramientas de interfaz en Consola, entre estos esta:

- Nextpnr-xilinx (Generación del archivo .xdc)
- Project X-Ray: Fasm2frames y xc7frames2bit (Generacion Bitstream)
- openFPGALoader (Inyectar el archive “.bit” en la placa booleana)

Procedimiento de desarrollo

Lo primero que hicimos fue crear el módulo “Calculator” Especificando los inputs y outputs, los cuales serán utilizados para los botones, especificamos los switches y los botones a utilizar.

```
1 // Calculadora para laboratorio 5
2 module Calculator (
3     ...input clk,
4     ...input [15:0] sw,
5     ...input [3:0] btn,
6     ...output [3:0] D0_AN,
7     ...output [7:0] D0_SEG,
8     ...output [3:0] D1_AN,
9     ...output [7:0] D1_SEG
10 );
11
12 ...wire [ 7:0] A = sw[7:0];
13 ...wire [ 7:0] B = sw[15:8];
14
```

También especificamos los bits los cuales vamos a utilizar para mostrar en la pantalla de 7 segmentos.

```
15 ...reg [15:0] result;
16 ...always @(*) begin
17     ...if (btn[0]) result = A + B; // Add operation
18
19     ...else if (btn[1]) result = {8'h00, A} - {8'h00, B}; // Subtract operation ; Wraps if B > A
20
21     ...else if (btn[2]) result = A * B; // Multiply operation, fits fully in 16 bits
22
23     ...else result = 16'h0000;
24 ...end
```

Hacemos una especie de condicional, la cual verifica si el usuario este sumando, restando o multiplicando.

...

```

26 ...function [7:0] hex_to_seg;
27 ...input [3:0] val;
28 ...begin
29 ...case (val)
30 ...4'h0: hex_to_seg = 8'b1100_0000;
31 ...4'h1: hex_to_seg = 8'b1111_1001;
32 ...4'h2: hex_to_seg = 8'b1010_0100;
33 ...4'h3: hex_to_seg = 8'b1011_0000;
34 ...4'h4: hex_to_seg = 8'b1001_1001;
35 ...4'h5: hex_to_seg = 8'b1001_0010;
36 ...4'h6: hex_to_seg = 8'b1000_0010;
37 ...4'h7: hex_to_seg = 8'b1111_1000;
38 ...4'h8: hex_to_seg = 8'b1000_0000;
39 ...4'h9: hex_to_seg = 8'b1001_0000;
40 ...4'ha: hex_to_seg = 8'b1000_1000;
41 ...4'hb: hex_to_seg = 8'b1000_0011;
42 ...4'hc: hex_to_seg = 8'b1100_0110;
43 ...4'hd: hex_to_seg = 8'b1010_0001;
44 ...4'he: hex_to_seg = 8'b1000_0110;
45 ...4'hf: hex_to_seg = 8'b1000_1110;
46 ...default: hex_to_seg = 8'b0000_0000;
47 ...endcase
48 ...end
49 ...endfunction

```

Hacemos una función la cual convierte los 4 bits hexadecimal en un patrón de 8 bits el cual será utilizado para imprimirse en la pantalla de 7 segmentos. Los valores van desde 0 hasta F en sistema hexadecimal.

```

51 ...// Now, we will refrest the counter for digit multiplexing.
52 ...reg [15:0] refresh_counter = 0;
53 ...always @(posedge clk) begin
54 ...refresh_counter <= refresh_counter + 1'b1;
55 ...end
56 ...wire [1:0] digit_sel = refresh_counter[15:14];

```

Ahora vamos a manejar las pantallas de 7 segmentos, haciendo una función la cual refresque la pantalla y simule “estabilidad” de imagen. Esto es para evitar bugs visuales.

Luego hacemos un `case` para los distintos escenarios el cual la pantalla podría estar

```

58 ...// Print in the first row
59 ...reg [7:0] d0_seg_r;
60 ...reg [3:0] d0_an_r;
61 ...always @(*) begin
62     ...case (digit_sel)
63     ...2'b00: begin
64         ...d0_an_r = 4'b1110; ...// AN0
65         ...d0_seg_r = hex_to_seg(B[3:0]);
66     ...end
67     ...2'b01: begin
68         ...d0_an_r = 4'b1101; ...// AN1
69         ...d0_seg_r = hex_to_seg(B[7:4]);
70     ...end
71     ...2'b10: begin
72         ...d0_an_r = 4'b1011; ...// AN2
73         ...d0_seg_r = hex_to_seg(A[3:0]);
74     ...end
75     ...2'b11: begin
76         ...d0_an_r = 4'b0111; ...// AN3
77         ...d0_seg_r = hex_to_seg(A[7:4]);
78     ...end
79     ...endcase
80 ...end
81 ...assign D0_AN = d0_an_r;
82 ...assign D0_SEG = d0_seg_r;

```

Y de paso, asignamos los valores a la pantalla con ``assign``. Repetimos el paso de asignar y el ``case`` con la otra sección de la pantalla de 7 segmentos.

```

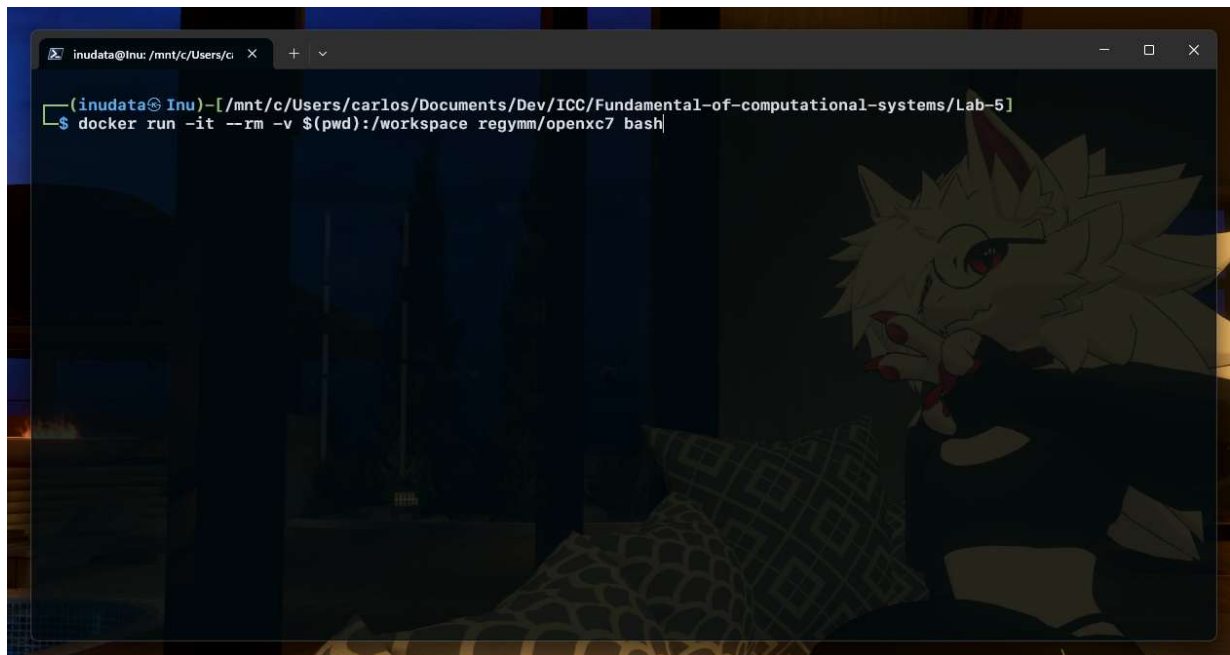
84 ...// Print in the second row.
85 ...reg [7:0] d1_seg_r;
86 ...reg [3:0] d1_an_r;
87 ...always @(*) begin
88     ...case (digit_sel)
89     ...2'b00: begin
90         ...d1_an_r = 4'b1110; ...// AN0
91         ...d1_seg_r = hex_to_seg();
92     ...end
93     ...2'b01: begin
94         ...d1_an_r = 4'b1101; ...// AN1
95         ...d1_seg_r = hex_to_seg();
96     ...end
97     ...2'b10: begin
98         ...d1_an_r = 4'b1011; ...// AN2
99         ...d1_seg_r = hex_to_seg();
100    ...end
101    ...2'b11: begin
102        ...d1_an_r = 4'b0111; ...// AN3
103        ...d1_seg_r = hex_to_seg();
104    ...end
105    ...endcase
106 ...end
107
108 ...assign D1_AN = d1_an_r;
109 ...assign D1_SEG = d1_seg_r;
110
111 endmodule

```

Luego de programar toda la lógica, el siguiente paso es compilar, crear el archivo `.bit` y probar el código con la placa booleana.

Como hicimos en el laboratorio pasado, utilizaremos Docker para correr una serie de comandos para crear el archivo compilado `.bit`

Primero que todo, inicializamos Docker, abriendo la aplicación de escritorio y luego la terminal. En la terminal, iniciamos el subsistema de Linux que Docker utiliza como defecto, en mi caso Kali-Linux, y corremos el siguiente comando:



Luego de inicializar Docker, nos dirigimos al directorio `'workspace'`, utilizando el comando en la terminal `'cd'`. Luego de entrar en Docker, corremos el siguiente comando, utilizando las herramientas mencionadas en el marco teórico.

```
yosys -p "synth_xilinx -flatten -abc9 -arch xc7 -top calculator -json calculator.json" calculator.v
nextpnr-xilinx --chipdb xc7s50.bin --xdc calculator.xdc --json calculator.json --fasm calculator.fasm
fasm2frames --part xc7s50csga324-1 --db-root /nextpnr-xilinx/xilinx/external/prjxray-db/spartan7 calculator.fasm
> calculator.frames
xc7frames2bit --part_file /nextpnr-xilinx/xilinx/external/prjxray-db/spartan7/xc7s50csga324-1/part.yaml --
part_name xc7s50csga324-1 --frm_file calculator.frames --output_file calculator.bit
```

Luego de correr la serie de comandos para compilar, vamos a salir de Docker con `'exit'`, y correr el siguiente comando en WSL:

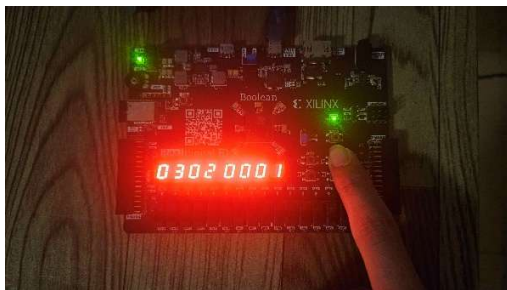
```
openFPGAloader -c ft2232 calculator.bit
```

Este inyectará el `.bit` en la placa booleana.

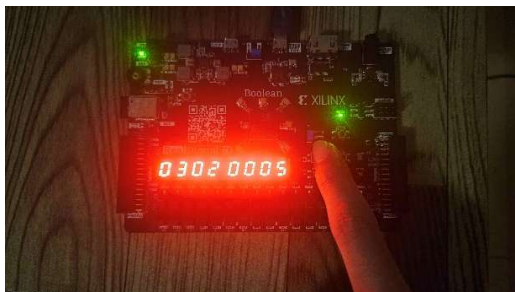
Conclusión

Este proyecto nos ha ayudado a comprender las operaciones aritméticas en “Verilog” y como funciona en su totalidad en bajo nivel estas operaciones, es decir, sumar o multiplicar en un nivel de hardware el cual es casi táctil. El comprender la comunicación de la maquina a base de operaciones lógicas, y a base también de la compilación de valores casi humanos como es el lenguaje “Verilog”, pero siendo igual de simbólico de muchos lenguajes de alto nivel como C, nos ayuda a formar bases de conocimiento para áreas que puede ser la arquitectura y desarrollo de procesadores gráficos, o unidades de procesamiento (CPU), los cuales pueden llegar a requerir una programación directa y optimizada como lo es “Verilog”.

Resta:



Suma:



Multipliación:

